

Restoring M1/M8 and Eager/Compiled Agreement on RDNA4

Terrydaktal · 17 September 2026 · Technical report, version 5

Abstract

A pinned Radiance/vLLM inference stack produced different target logits when the same saved token history was processed one position at a time (M1) or eight positions at a time under D7 speculative verification (M8). The investigation localized numerical differences in GDN prefill and decode, convolution, gated normalization, attention, residual normalization and the BF16 vocabulary head. The Fix 1 compiled implementations agree on top-1/10/20 membership, ordering, retained scores, inclusive boundary ties and full-vocabulary logit hashes at 10,000 decode positions and 23 initial-prefill predictions. Four subsequent performance changes reduced a repaired verification round from approximately 76 ms to 60 ms in the measured 60K-input control.

A later eager/compiled investigation isolated intermediate BF16 rounding and native RoPE multiplication differences. Aligning both produced identical full logit vectors at 320 decode positions and the final prefill prediction, with identical captured inputs and outputs at all 465 matched activation boundaries. This newer eager/compiled M8 result uses a separate 320-position replay; the 10K comparison has not been repeated under the new rounding settings.

This is an empirical result on one pinned configuration; it does not establish arbitrary-input equivalence or improved task accuracy.

The work consists of **two major fixes**:

Revision	What changed	Comparison it repairs
Original	Pinned backend before these repairs	Baseline
Fix 1	Align M1/M8 arithmetic and state transitions, then recover performance	Compiled M8 versus compiled M1
Final: Fix 1 + Fix 2	Preserve BF16 intermediate rounding in compiled execution and use nearest-even RoPE products in eager execution	Compiled M8 versus eager M8

1. Configuration and reference

- One AMD Radeon AI PRO R9700, gfx1201, TP1.
- Radiance 1.0.16 / vLLM 0.28, PyTorch 2.12.0, ROCm 7.14, Triton 3.7.1.
- Qwen3.8-27B-Uncensored-MXFP4-awq; 64 layers, including 48 GDN and 16 attention layers; hidden width 5,120; vocabulary size 248,320.
- The target vocabulary head uses the full original BF16 weights in both arms. INT2 candidate selection is excluded from the equality claim. Other model weights, activation quantization and cache formats remain part of the pinned quantized backend; this is not an unquantized BF16-model comparison.
- The compiled comparisons use Inductor and piecewise GPU graphs, with capture sizes [1, 2, 4, 8]. Actual target graph replay is audited.
- The numerical reference is the explicitly selected serial implementation. Repairs also affect M1/preparation paths, so repaired M1 is measured afresh.

The 10K corpus comprises 23 saved Pi continuations with 57,008–65,527-token prefixes. Both arms consume identical saved token histories at aligned logical positions. Each M8 group processes eight positions with all seven supplied proposals accepted. The denominator is 10,000 processed decode positions, excluding intermediate-prefill head calls. Final initial-prefill predictions have a separate denominator of 23. No generated tools execute during replay.

2. Agreement definitions

For each position t , let $z1(t)$ and $z8(t)$ be the full vocabulary logit vectors. T_k sorts logits by decreasing score and then increasing token ID for ties.

```

set_agreement(k) = count(set(T_k(z1)) == set(T_k(z8))) / N
order_agreement(k) = count(T_k(z1) == T_k(z8)) / N
mean_shared(k) = sum(|set(T_k(z1)) intersect set(T_k(z8))|) / N

```

Retained-score equality, inclusive boundary-tie membership and full-vector SHA-256 equality are separate checks. Matching hashes are strong empirical evidence, not a collision-free mathematical proof. Agreement measures execution consistency; it is not a percentage of questions answered correctly.

3. Top-1/10/20 and ordering results

Fix 1, compiled and optimized: full 10K corpus

Prediction	Same token set	Same ordering	Mean shared tokens
Top 1	10,000 / 10,000 (100%)	10,000 / 10,000 (100%)	1 / 1
Top 10	10,000 / 10,000 (100%)	10,000 / 10,000 (100%)	10 / 10
Top 20	10,000 / 10,000 (100%)	10,000 / 10,000 (100%)	20 / 20

Retained scores, inclusive tie sets and full-vocabulary hashes also match at all 10,000 positions. All measures separately match for **23/23 initial-prefill predictions**. Both workers exited successfully. The completion audit checked all per-continuation receipts, fixture identities, widths, aligned positions, runtime receipts and the recomputed aggregate.

Fresh four-way comparison: 320 positions

Seven fresh native runs used the same saved 60,000-token Pi prefix and 320 aligned decode positions. Each M1/M8 pair uses its own revision's M1. Compiled arms ran with Inductor and piecewise GPU graphs; eager arms explicitly disabled compilation and graphs.

Reading paired values: first = same token set (any order); second = same ranked order. 320/320; 320/320 means both checks matched at all 320 tested positions.

Compared implementations	Top-1 set/order	Top-10 set/order	Top-20 set/order	Full vectors exact
Original compiled M8 vs original compiled M1	315/320; 315/320	157/320; 18/320	83/320; 1/320	0/320
Fix 1 compiled M8 vs Fix 1 compiled M1	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320
Fix 1 compiled M8 vs Fix 1 eager M8	319/320; 319/320	146/320; 14/320	66/320; 0/320	0/320
Final compiled M8 vs final eager M8 (Fix 1 + Fix 2)	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320

These are **whole-model** results. The isolated stage comparisons in section 4 use common correct inputs to prevent upstream errors contaminating the stage being measured. [Run identities and complete counts](#).

Historical unfixed eager baseline: same 10K corpus

Prediction	Same token set	Same ordering	Mean shared tokens
Top 1	9,852/10,000 (98.52%)	9,852/10,000 (98.52%)	0.9852 / 1
Top 10	4,857/10,000 (48.57%)	799/10,000 (7.99%)	9.3487 / 10
Top 20	2,270/10,000 (22.70%)	9/10,000 (0.09%)	18.6781 / 20

These historical results use eager execution. They must not be presented as an unfixed **compiled** 10K measurement. The Fix 1 compiled run uses a fresh compiled M1 reference, not this saved eager reference.

4. Compiled stage timings and isolated correctness measurements

Both timing profiles use the same 60,000-input-token Pi fixture and the optimized compiled configuration: **enforce_eager=false, Inductor, PIECEWISE GPU graphs, capture sizes [1, 2, 4, 8]**. The actual traces contain **65 target GPU graph launches in every captured round**, in both arms. Runtime receipts are recorded in [final-study-profile-audit.json](#). These are not eager timings. The full BF16 target head is used on both sides.

The fresh profiles contain one incomplete target-kernel inventory in each arm: original round 8 and final round 7 (one-based). The stage table uses the **six paired complete rounds, 1-6**, selected by inventory completeness rather than timing. Every recorded dispatch, including excluded rounds, remains in the evidence. [Profile audit](#).

All times are milliseconds per verification round: summed GPU dispatch durations for all layer instances, divided by six. The 256 projection dispatches per round are assigned to their four operations in each of 64 layers. The approximately 25 ms gate/up figure is the joint MLP projection across all 64 layers; its individual layer contributions follow.

The four correctness columns report **vocabulary top-20 set/order agreement on 320 positions**, with each layer instance given common correct inputs. Only one instance is substituted at a time before the native reference remainder produces logits. A position passes an aggregated stage only when every instance passes. Exact local outputs and state can reuse the already validated reference remainder; shared implementations across revisions are identified explicitly. These are not rankings of intermediate activation coordinates. The all-layer requirement makes these rates stricter than a single whole-model comparison.

“Performance” marks overhead removed from the initial repaired implementation; it does not imply that a stage is faster than the original unrepaired M8. A Fix 1 label records alignment to the selected serial arithmetic. Some original compiled M1/M8 stages already agree with each other on this sample; their actual agreement counts remain shown rather than being labelled as failures.

[Per-layer comparison counts](#) retain the individual results behind the aggregate table, including top-1/10/20 and full logit-vector agreement.

Original comparisons use the original M1 and M8 stage implementations; Fix 1 comparisons use the repaired implementations. Every column uses the same final-correct captured inputs. Thus these isolated rates diagnose individual stages; they are not the whole-model rates from section 3.

Compiled diagnostic replay disables graph replay to expose individual calls; its ordinary outputs must first match the graph-enabled release. **Diagnostic replay durations are excluded from this timing table**. The original and final columns below use fresh graph-enabled release profiles.

Reading each top-20 pair: first = the same 20 tokens, in any order; second = those 20 tokens in exactly the same ranked order. 320/320; 320/320 means both checks passed at all 320 tested positions. A position passes a stage only when every tested layer instance agrees.

Compiled stage	Correctness fix?	Old compiled M8	Final fixed compiled M8 ms	Change ms	Old compiled M8 vs old compiled M1 Top-20 set/order	Fix 1 compiled M8 vs Fix 1 compiled M1 Top-20 set/order	Fix 1 compiled M8 vs Fix 1 eager M8 Top-20 set/order	Final compiled M8 vs final eager M8 Top-20 set/order	Timing explanation
Embedding + first input normalization	Fix 1: normalization; embedding unchanged	0.004	0.008	+0.004	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Preserve the reference normalization rounding; the original embedding/norm fusion is indivisible.
Layer input residual/ normalization	Fix 1 + performance	0.155	0.346	+0.191	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Preserve reduction and rounding; retain FP32 residual sums in registers.
GDN input activation FP8 quantization	No correctness repair	0.127	0.127	0.000	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same quantization kernel and call count; the small timing delta is unassigned.
GDN input projection	No correctness repair	3.875	3.873	-0.002	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Unchanged MXFP4 projection arithmetic; timing differences are observations, not a projection optimization.
GDN layout/copies and buffer initialization	Performance only	0.284	0.332	+0.047	Within convolution/ recurrence cuts	Within convolution/ recurrence cuts	Within convolution/ recurrence cuts	Within convolution/ recurrence cuts	Preserve packed QKV views; remove split materializations and repacking. Remaining copies are included.
GDN convolution	Fix 1 + performance	0.494	0.226	-0.268	9/320; 0/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Match serial product/accumulation order and rolling history; packed transport removes surrounding copies.
GDN recurrence and gates	Fix 1	1.162	1.219	+0.057	13/320; 0/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Match gate precision, reduction order, recurrent-state transition and output rounding.
GDN output gated normalization	Fix 1 + performance	0.138	0.097	-0.041	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Keep the serial row tile while processing independent rows concurrently; original fused constituents remain grouped.
GDN output activation FP8 quantization	No correctness repair	0.128	0.131	+0.003	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same quantization kernel and call count; the small timing delta is unassigned.
GDN output projection	No correctness repair	1.884	1.884	0.000	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Unchanged MXFP4 arithmetic; no causal speedup claimed.
Attention input activation FP8 quantization	No correctness repair	0.043	0.042	-0.001	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same quantization kernel and call count; timing cause is not isolated.
Attention input projection	No correctness repair	1.099	1.099	0.000	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Unchanged QKV MXFP4 projection; no causal speedup claimed.
Attention Q/K normalization, RoPE and layout	Fix 1: normalization; Fix 2: RoPE rounding	0.079	0.236	+0.157	320/320; 320/320	320/320; 320/320	14/320; 0/320	320/320; 320/320	Preserve serial normalization and BF16 RoPE product rounding. Fused original constituents share one timing.
Attention KV write	No correctness repair	0.046	0.047	+0.001	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same cache-write kernel and KV format; timing cause is not isolated.
Attention decode	Fix 1 + performance	4.224	5.285	+1.061	22/320; 0/320 †	320/320; 320/320 †	320/320; 320/320 †	320/320; 320/320 †	Preserve each query's causal tile/softmax decisions. Share KV reads within one or two tile-aligned query groups; two groups repeat context work.
Attention split-KV merge	Fix 1	0.138	0.120	-0.018	22/320; 0/320 †	320/320; 320/320 †	320/320; 320/320 †	320/320; 320/320 †	Use each query's serial split/merge arithmetic; the observed reduction has not been isolated from the decode change.
Attention output gating	Fix 2: intermediate rounding	0.028	0.032	+0.004	320/320; 320/320	320/320; 320/320	20/320; 0/320	320/320; 320/320	Preserve the BF16 sigmoid result before multiplying the output gate.
Attention output activation FP8 quantization	No correctness repair	0.046	0.046	0.000	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same quantization kernel and call count; timing cause is not isolated.
Attention output projection	No correctness repair	0.557	0.521	-0.035	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Unchanged output MXFP4 projection; no causal speedup claimed.
Post-attention/GDN residual/ normalization	Fix 1 + performance	0.142	0.353	+0.210	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Preserve serial reduction/rounding and keep residual values in registers.
MLP gate/up input FP8 quantization	No correctness repair	0.165	0.167	+0.002	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same quantization kernel and call count; timing cause is not isolated.
MLP gate/up projection	No correctness repair	25.271	25.437	+0.166	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	One joint gate/up GEMM per layer, 64 per round. The per-layer table subdivides this total; gate and up have no separate measured durations.
MLP SiLU and gating	Fix 2: intermediate rounding	0.156	0.189	+0.033	320/320; 320/320	320/320; 320/320	15/320; 0/320	320/320; 320/320	Preserve the BF16 SiLU result before multiplication; retain one fused pointwise launch.
MLP down input FP8 quantization	No correctness repair	0.297	0.297	+0.001	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Same quantization kernel and call count; timing cause is not isolated.
MLP down projection	No correctness repair	5.147	5.143	-0.004	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Unchanged MXFP4 down projection; no causal speedup claimed.
Final normalization/layout	Fix 1	0.002	0.006	+0.003	320/320; 320/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Preserve final-normalization reduction and the BF16 rounding of its retained residual sum; isolated check includes that fused addition.
Full BF16 target head	Fix 1 + performance	4.024	4.140	+0.116	320/320; 319/320	320/320; 320/320	320/320; 320/320	320/320; 320/320	Interleave two arithmetic-preserving M4 groups in one HIP launch, removing duplicated launch and concatenation overhead.
Drafter	No target correctness repair	6.328	6.518	+0.190	N/A: not a target prediction stage	N/A: not a target prediction stage	N/A: not a target prediction stage	N/A: not a target prediction stage	Unchanged proposal model; its timings and predictions are not target-M1 equivalence measurements.
Other GPU bookkeeping	Not an isolated numerical stage	0.551	0.560	+0.009	N/A: not a target prediction stage	N/A: not a target prediction stage	N/A: not a target prediction stage	N/A: not a target prediction stage	Sampling/state bookkeeping outside the model scopes. Exact semantic attribution is unavailable; each kernel remains listed below.

† Attention decode and split-KV merge have separate GPU timings but one native numerical interface, so their correctness cells report the joint comparison. GDN transport copies are included in the convolution/recurrence state checks; they do not have an independent vocabulary-prediction result. Drafter and bookkeeping work are outside the target-stage comparison.

Target-body subtotal: **45.693 ms old; 47.263 ms fixed**. Total recorded GPU kernel duration: **56.596 ms old; 58.481 ms fixed**. These are subtotals, not extra stages. Kernel durations can overlap and exclude host/queue gaps; they are not wall-clock round latency.

Interpreting the timing differences

The FP8 quantizer is unchanged and runs 256 times per round in both versions. Its fresh totals are **0.806 ms original and 0.810 ms final**. The earlier large quantization timing difference does not reproduce in this comparison.

Attention decode averages **4.224 ms original and 5.285 ms final**. Final rounds split into approximately 3.55 ms and 7.02 ms groups. The repaired kernel uses one or two groups of queries aligned to 16-token tiles to preserve each query's M1 causal and softmax decisions; a second group repeats context work. The split-KV merge is timed separately: **0.138 ms original and 0.120 ms final**.

An unchanged stage's timing delta alone does not establish a performance fix.

Every layer, including the 64 contributions to the gate/up total

Layer numbers are zero-based. "Other three projections" means the input, attention/GDN output, and MLP down projections; all individual semantic stage values are also retained in `stage-times.json`. Non-projection work includes normalization, gating, quantization, attention/GDN operations and copies.

Expand all 64 layers

Layer (zero-based)	Type	All layer work old ms	Final fixed ms	Gate/up old ms	Final fixed ms	Other three projections old ms	Final fixed ms	Non-projection old ms	Final fixed ms
0	GDN	0.6255	0.6298	0.3712	0.3704	0.1946	0.1945	0.0597	0.0649
1	GDN	0.6274	0.6246	0.3712	0.3664	0.1986	0.1983	0.0577	0.0599
2	GDN	0.6321	0.6298	0.3716	0.3692	0.1989	0.1986	0.0616	0.0620
3	Attention	0.8453	0.9203	0.3747	0.3743	0.1831	0.1801	0.2876	0.3659
4	GDN	0.6372	0.6387	0.3797	0.3784	0.1993	0.1986	0.0582	0.0617
5	GDN	0.6423	0.6460	0.3828	0.3844	0.1996	0.1989	0.0599	0.0627
6	GDN	0.6501	0.6501	0.3868	0.3858	0.2003	0.1995	0.0630	0.0647
7	Attention	0.8685	0.9506	0.3884	0.3909	0.1833	0.1807	0.2968	0.3790
8	GDN	0.6468	0.6532	0.3876	0.3905	0.1991	0.1987	0.0600	0.0641
9	GDN	0.6481	0.6572	0.3875	0.3924	0.2002	0.2004	0.0604	0.0644
10	GDN	0.6543	0.6575	0.3902	0.3913	0.2000	0.2008	0.0641	0.0653
11	Attention	0.8717	0.9490	0.3912	0.3899	0.1826	0.1802	0.2979	0.3788
12	GDN	0.6526	0.6541	0.3912	0.3905	0.1999	0.2002	0.0615	0.0634
13	GDN	0.6521	0.6551	0.3904	0.3897	0.2005	0.2009	0.0612	0.0645
14	GDN	0.6536	0.6587	0.3881	0.3939	0.2012	0.1995	0.0643	0.0653
15	Attention	0.8739	0.9551	0.3909	0.3940	0.1839	0.1808	0.2991	0.3803
16	GDN	0.6519	0.6576	0.3909	0.3931	0.1998	0.1999	0.0612	0.0646
17	GDN	0.6531	0.6607	0.3911	0.3952	0.2009	0.2004	0.0610	0.0651
18	GDN	0.6587	0.6637	0.3910	0.3972	0.2016	0.2004	0.0662	0.0661
19	Attention	0.8778	0.9606	0.3926	0.3961	0.1831	0.1815	0.3020	0.3831
20	GDN	0.6580	0.6628	0.3955	0.3949	0.2005	0.2006	0.0620	0.0673

Layer (zero-based)	Type	All layer work old ms	Final fixed ms	Gate/up old ms	Final fixed ms	Other three projections old ms	Final fixed ms	Non- projection old ms	Final fixed ms
21	GDN	0.6583	0.6637	0.3971	0.3955	0.1999	0.2021	0.0614	0.0660
22	GDN	0.6630	0.6630	0.3977	0.3966	0.1999	0.2009	0.0654	0.0654
23	Attention	0.8782	0.9618	0.3938	0.3966	0.1831	0.1812	0.3014	0.3839
24	GDN	0.6550	0.6633	0.3926	0.3970	0.2005	0.2009	0.0619	0.0653
25	GDN	0.6546	0.6629	0.3926	0.3959	0.2005	0.2013	0.0615	0.0658
26	GDN	0.6622	0.6633	0.3948	0.3968	0.2016	0.2012	0.0659	0.0653
27	Attention	0.8835	0.9642	0.3948	0.3978	0.1847	0.1814	0.3040	0.3850
28	GDN	0.6579	0.6658	0.3946	0.3989	0.2016	0.1999	0.0617	0.0670
29	GDN	0.6606	0.6682	0.3970	0.4009	0.2007	0.2013	0.0628	0.0661
30	GDN	0.6652	0.6682	0.3983	0.4019	0.2010	0.2005	0.0660	0.0659
31	Attention	0.8834	0.9707	0.3966	0.4007	0.1844	0.1811	0.3025	0.3889
32	GDN	0.6588	0.6671	0.3970	0.3999	0.2010	0.2004	0.0608	0.0668
33	GDN	0.6600	0.6675	0.3965	0.4013	0.2013	0.2005	0.0622	0.0656
34	GDN	0.6646	0.6704	0.3981	0.4022	0.2009	0.2008	0.0656	0.0674
35	Attention	0.8900	0.9715	0.4003	0.4021	0.1836	0.1815	0.3062	0.3880
36	GDN	0.6646	0.6680	0.4001	0.4024	0.2017	0.1997	0.0628	0.0660
37	GDN	0.6616	0.6701	0.3983	0.4033	0.2010	0.2013	0.0622	0.0655
38	GDN	0.6671	0.6717	0.4007	0.4043	0.2011	0.2009	0.0654	0.0665
39	Attention	0.8897	0.9784	0.4000	0.4058	0.1831	0.1815	0.3066	0.3911
40	GDN	0.6629	0.6723	0.3994	0.4064	0.2007	0.2006	0.0627	0.0652
41	GDN	0.6629	0.6730	0.3993	0.4057	0.2008	0.2006	0.0627	0.0666
42	GDN	0.6651	0.6751	0.3992	0.4062	0.2005	0.2028	0.0655	0.0661
43	Attention	0.8903	0.9743	0.4013	0.4038	0.1836	0.1814	0.3053	0.3892
44	GDN	0.6671	0.6686	0.4023	0.4022	0.2015	0.1999	0.0632	0.0665
45	GDN	0.6669	0.6672	0.4030	0.4014	0.2007	0.2005	0.0632	0.0653
46	GDN	0.6671	0.6699	0.4009	0.4026	0.2004	0.2012	0.0658	0.0661
47	Attention	0.8922	0.9736	0.4026	0.4041	0.1834	0.1819	0.3062	0.3877
48	GDN	0.6687	0.6745	0.4040	0.4057	0.2018	0.2009	0.0628	0.0679
49	GDN	0.6713	0.6746	0.4064	0.4075	0.2016	0.2004	0.0633	0.0667
50	GDN	0.6709	0.6721	0.4037	0.4043	0.2009	0.2011	0.0663	0.0667
51	Attention	0.8912	0.9744	0.4003	0.4037	0.1844	0.1810	0.3065	0.3898
52	GDN	0.6629	0.6704	0.3994	0.4040	0.2005	0.2001	0.0630	0.0663
53	GDN	0.6618	0.6711	0.3997	0.4047	0.2004	0.2008	0.0617	0.0656
54	GDN	0.6692	0.6741	0.4012	0.4058	0.2018	0.2018	0.0663	0.0665
55	Attention	0.8885	0.9742	0.3995	0.4032	0.1838	0.1811	0.3051	0.3899
56	GDN	0.6586	0.6692	0.3974	0.4028	0.1993	0.2004	0.0619	0.0660
57	GDN	0.6636	0.6692	0.3996	0.4014	0.2012	0.2019	0.0628	0.0659
58	GDN	0.6692	0.6695	0.4010	0.4021	0.2019	0.2012	0.0664	0.0662

Layer (zero-based)	Type	All layer work old ms	Final fixed ms	Gate/up old ms	Final fixed ms	Other three projections old ms	Final fixed ms	Non-projection old ms	Final fixed ms
59	Attention	0.8904	0.9620	0.4009	0.4019	0.1837	0.1820	0.3059	0.3781
60	GDN	0.6652	0.6723	0.4008	0.4053	0.2015	0.2004	0.0630	0.0667
61	GDN	0.6649	0.6802	0.4011	0.4099	0.2011	0.2021	0.0627	0.0682
62	GDN	0.6688	0.6779	0.4018	0.4077	0.2008	0.2016	0.0661	0.0686
63	Attention	0.8917	0.9819	0.4010	0.4064	0.1840	0.1824	0.3067	0.3932

Every recorded compiled kernel within each semantic stage

Calls are totals over the six retained rounds; times are per-round means. Compiler-fused operations remain indivisible. The tables do not invent times for separate constituents of a single GPU dispatch. Individual dispatch durations, layer IDs and round IDs are retained in the two evidence/final-study-*.dispatches.json exports; they contain no token IDs, raw activations, absolute timestamps or process identifiers.

Expand the complete kernel breakdown

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
drafter / Drafter Cijk_Alik_Bljk_BBS_BH_Bias_HA_S_SAV_UserArgs_MT16x16x32_MI16x16x1_SN_LDSB1_AFC1_AG0_AGGSUA0_AGNTAB0_AFEM1_AFEM1_ASEM1_CD1_1_CLR0_CLS0_CADS0_DTLA0_DTLB0_DTLMO_DTVA0_DTVB1_DTVMSA0_DTVMSB0_DTVSM0_DPLB0_EPS0_ELFLR0_EMLLn1_FDSI0_GRPm1_GRVWA8_GRVWB8_GSUAMB_GLS0_HPLR0_ISA1201_ICIW0_IU1_K1_LDSTI0_LBSPPA128_LBSPPB0_LBSPPMXA0_LBSPPMXSBO_LBSPPM0_LPA16_LPB0_LPMXA0_LPMXSBO_LPM0_LRVW8_LWPMn1_MIAV1_MIWT1_1_MXLIBL_MXSFNS_MO40_MGRIPM1_NTN1_NTA0 NTB0_NTC0_NTD0_NTE0_NTMXA0_NTMXSBO_NTM0_NTWS0_NVn1_NVA0_NVBO_NVC0_NVD0_NVE0_NVMXA0_NVMXSBO_NVM0_NVWS0_NEPBS0_NLCA1_NLCB2_ONLL1_PAP0_PGL0_PGR1_PLR1_PKA0_SGROB0_SIA3_SS0_SPO0_SRVW0_SSO0_SVW8_SK0_SKFTR0_SKFDP00_SKXCCM0_SNLL0_SIP1_SGRO0_TDMI0_TDMIM0_TDMS0_TIN0_Thn1_THA0_THB0_THC0_THD0_THE0_THMXSA0_THMXSBO_THM0_THWS0_TLDS1_TLDSM1_ULSGRO0_USL1_USLMX0_UIOFGRO0_UPLRP0_USFGR0n1_USI0_VSn1_VWA1_VWB1_WSGRA0_WSGRB0_WS32_WG16_2_1.kd	6 / 6	0.007179	0.007186
drafter / Drafter Cijk_Alik_Bljk_BBS_BH_Bias_HA_S_SAV_UserArgs_MT16x32x256_MI16x16x1_SN_LDSB0_AFC1_AG0_AGGSUA0_AGNTAB0_AFEM1_AFEM1_ASEM1_CD1_1_CLR1_CLS0_CADS0_DTLA0_DTLB0_DTLMO_DTVA0_DTVB1_DTVMSA0_DTVMSB0_DTVSM0_DPLB0_EPS1_ELFLR0_EMLLn1_FDSI0_GRPm1_GRVWA8_GRVWB8_GSUAMB_GLS0_HPLR0_ISA1201_ICIW0_IU1_K1_LDSTI0_LBSPPA512_LBSPPB0_LBSPPMXA0_LBSPPMXSBO_LBSPPM0_LPA16_LPB0_LPMXA0_LPMXSBO_LPM0_LRVW8_LWPMn1_MIAV1_MIWT1_1_MXLIBL_MXSFNS_M040_MGRIPM1_NTN1_NTA0 NTB0_NTC0_NTD0_NTE0_NTMXA0_NTMXSBO_NTM0_NTWS0_NVn1_NVA0_NVBO_NVC0_NVD0_NVE0_NVMXA0_NVMXSBO_NVM0_NVWS0_NEPBS0_NLCA1_NLCB16_ONLL0_PAP0_PGL0_PGR1_PLR1_PKA0_SGROB0_SIA3_SS0_SPO0_SRVW0_SSO0_SVW8_SK0_SKFTR0_SKFDP00_SKXCCM0_SNLL0_SIP1_SGRO0_TDMI0_TDMIM0_TDMS0_TIN0_Thn1_THA0_THB0_THC0_THD0_THE0_THMXSA0_THMXSBO_THM0_THWS0_TLDS1_TLDSM1_ULSGRO0_USL1_USLMX0_UIOFGRO0_UPLRP0_USFGR0n1_USI0_VSn1_VWA1_VWB1_WSGRA0_WSGRB0_WS32_WG16_4_1.kd	6 / 6	0.177347	0.177167
drafter / Drafter __amd_rocclr_copyBuffer.kd	6 / 6	0.001919	0.002012
drafter / Drafter __amd_rocclr_fillBufferAligned.kd	18 / 18	0.007077	0.006977
drafter / Drafter _cache_draft_logits_kernel.kd	6 / 6	0.002359	0.002292
drafter / Drafter _draft_head_int2.kd	6 / 6	0.824683	0.840683
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLOCK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_17408_EVEN_K_1_GRID_MN_40_cache_modifier_NONE.kd	30 / 30	0.730700	0.739805

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLOCK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_25600_EVEN_K_1_GRID_MN_40_cache_modifier_NONE.kd	6 / 6	0.218080	0.228567
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLOCK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_4096_EVEN_K_1_GRID_MN_40_cache_modifier_NONE.kd	30 / 30	0.202844	0.203789
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLOCK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_5120_EVEN_K_1_GRID_MN_272_cache_modifier_NONE.kd	30 / 30	1.446181	1.447936
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLOCK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_5120_EVEN_K_1_GRID_MN_48_cache_modifier_NONE.kd	30 / 30	0.273222	0.274569
drafter / Drafter _prepare_dflash_inputs_kernel.kd	6 / 6	0.007819	0.008032
drafter / Drafter _rerank_exact.kd	6 / 6	0.008606	0.008919
drafter / Drafter _selector_walk_kernel.kd	6 / 6	0.007866	0.008092
drafter / Drafter kernel_unified_attention.kd	30 / 30	1.852924	1.957785
drafter / Drafter reshape_and_cache_kernel_flash.kd	60 / 60	0.026984	0.027483
drafter / Drafter triton_per_fused_4.kd	6 / 6	0.001926	0.001986
drafter / Drafter triton_per_fused_8.kd	0 / 24	0.000000	0.007603
drafter / Drafter triton_per_fused_9.kd	24 / 0	0.007490	0.000000
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_preshuffle_gemm_squeeze_view_0.kd	30 / 30	0.009582	0.010142
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_preshuffle_gemm_squeeze_view_2.kd	6 / 6	0.002272	0.002086
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_preshuffle_gemm_squeeze_view_4.kd	54 / 54	0.014478	0.016224
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_view_0.kd	6 / 6	0.004099	0.005499
drafter / Drafter triton_poi_fused_0.kd	6 / 6	0.002686	0.002606
drafter / Drafter triton_poi_fused_10.kd	24 / 0	0.007503	0.000000
drafter / Drafter triton_poi_fused_5.kd	6 / 6	0.002232	0.002319
drafter / Drafter triton_poi_fused_9.kd	0 / 24	0.000000	0.009156
drafter / Drafter triton_poi_fused__to_copy_clamp_div_mul_preshuffle_gemm_silu_slice_squeeze_view_7.kd	30 / 0	0.015428	0.000000

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
drafter / Drafter triton_poi_fused__to_copy_clamp_div_preshuffle_gemm_squeeze_view_1.kd	30 / 30	0.011795	0.011862
drafter / Drafter triton_poi_fused__to_copy_clamp_div_preshuffle_gemm_squeeze_view_3.kd	6 / 6	0.002059	0.002146
drafter / Drafter triton_poi_fused__to_copy_clamp_div_preshuffle_gemm_squeeze_view_5.kd	54 / 54	0.016371	0.016211
drafter / Drafter triton_poi_fused_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_s elect_slice_unsqueeze_view_3.kd	54 / 54	0.019358	0.031305
drafter / Drafter triton_poi_fused_add_arange_bitwise_and_constant_pad_nd_ge_mul_rms_norm_select_slic e_unsqueeze_view_1.kd	6 / 6	0.002306	0.023693
drafter / Drafter triton_poi_fused_add_permute_unsqueeze_view_2.kd	6 / 6	0.001846	0.001846
drafter / Drafter triton_poi_fused_cat_expand_index_mul_slice_unsqueeze_view_1.kd	6 / 6	0.002546	0.002539
drafter / Drafter triton_red_fused__to_copy_abs_clamp_div_max_mul_preshuffle_gemm_silu_slice_squeeze_ view_6.kd	30 / 30	0.013422	0.018855
drafter / Drafter triton_red_fused__to_copy_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm _ge_mul_select_slice_unsqueeze_view_w4_gemm_2.kd	30 / 30	0.030876	0.032549
drafter / Drafter triton_red_fused__to_copy_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm _ge_mul_select_slice_unsqueeze_view_w4_gemm_7.kd	0 / 24	0.000000	0.027043
drafter / Drafter triton_red_fused__to_copy_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm _ge_mul_select_slice_unsqueeze_view_w4_gemm_8.kd	24 / 0	0.025756	0.000000
drafter / Drafter triton_red_fused__to_copy_embedding_mul_rms_norm_w4_gemm_0.kd	6 / 6	0.003846	0.004519
drafter / Drafter triton_red_fused_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_s elect_slice_unsqueeze_view_7.kd	0 / 6	0.000000	0.006599
drafter / Drafter triton_red_fused_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_s elect_slice_unsqueeze_view_8.kd	6 / 0	0.006719	0.000000
drafter / Drafter void at::native::(anonymous namespace)::CatArrayBatchedCopy_contig<at::native:: (anonymous namespace)::OpaqueType<2u>, unsigned int, 2, 128, 1>(at::native:: (anonymous namespace)::OpaqueType<2u>*, at::native::(anonymous namespace)::CatArrInputTensorMetadata<at::native::(anonymous namespace)::OpaqueType<2u>, unsigned int, 128, 1>, at::native::(anonymous namespace)::TensorSizeStride<unsigned int, 4u>, int, unsigned int) [clone .kd]	12 / 12	0.009285	0.009498
drafter / Drafter void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::lambda(int)#1>(int, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::lambda(int)#1)) [clone .kd]	6 / 6	0.003526	0.003532

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
drafter / Drafter void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<2>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<2>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}) [clone .kd]	6 / 6	0.002879	0.003066
drafter / Drafter void at::native::bitonicSortKVInPlace<2, -1, 16, 16, c10::BFloat16, long, at::native::GTOp<c10::BFloat16, true>, unsigned int>(at::cuda::detail::TensorInfo<c10::BFloat16, unsigned int>, unsigned int, unsigned int, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, at::native::GTOp<c10::BFloat16, true>) [clone .kd]	6 / 6	0.003232	0.003306
drafter / Drafter void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorB ase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorB ase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	6 / 6	0.005012	0.005039
drafter / Drafter void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIt eratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIt eratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	6 / 6	0.002586	0.002546
drafter / Drafter void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIt eratorBase&)::{lambda()#3}::operator()() const::{lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIt eratorBase&)::{lambda()#3}::operator()() const::{lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	6 / 6	0.003506	0.003692

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
drafter / Drafter void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 >>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 >>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	6 / 6	0.003146	0.003146
drafter / Drafter void at::native::mbtopk::computeBlockDigitCounts<c10::BFloat16, unsigned int, unsigned int, 2>(at::cuda::detail::TensorInfo<c10::BFloat16 const, unsigned int>, unsigned int, unsigned int, unsigned int, unsigned int, int, int, unsigned int, unsigned int, unsigned int*, short*) [clone .kd]	12 / 12	0.038345	0.041232
drafter / Drafter void at::native::mbtopk::computeBlockDigitCounts<float, unsigned int, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int*, unsigned int, unsigned int, int, int, unsigned int, unsigned int, unsigned int*, short*) [clone .kd]	24 / 24	0.055463	0.057470
drafter / Drafter void at::native::mbtopk::computeBlockwiseWithinKCounts<unsigned int, c10::BFloat16>(unsigned int*, short*, unsigned int*, unsigned int, int, bool, unsigned int*, c10::BFloat16*, unsigned int*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	12 / 12	0.022351	0.022578
drafter / Drafter void at::native::mbtopk::computeBlockwiseWithinKCounts<unsigned int, float>(unsigned int*, short*, unsigned int*, unsigned int, int, bool, unsigned int*, float*, unsigned int*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	24 / 24	0.028736	0.029063
drafter / Drafter void at::native::mbtopk::fill<unsigned int, unsigned int>(unsigned int*, unsigned int, unsigned int) [clone .kd]	12 / 12	0.003018	0.003005
drafter / Drafter void at::native::mbtopk::gatherTopK<c10::BFloat16, unsigned int, 2>(at::cuda::detail::TensorInfo<c10::BFloat16 const, unsigned int>, unsigned int, unsigned int, bool, unsigned int, unsigned int, at::cuda::detail::TensorInfo<c10::BFloat16, unsigned int>, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, unsigned int, unsigned int, c10::BFloat16*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	6 / 6	0.021039	0.021259
drafter / Drafter void at::native::mbtopk::gatherTopK<float, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int, bool, unsigned int, unsigned int, at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, unsigned int, unsigned int, float*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	6 / 6	0.008879	0.013026
drafter / Drafter void at::native::reduce_kernel<512, 1, at::native::ReduceOp<float, at::native::func_wrapper_t<float, at::native::sum_functor<float, float, float>::operator()(at::TensorIterator&)::{lambda(float, float)#1}>, unsigned int, float, 4, 4>>(at::native::ReduceOp<float, at::native::func_wrapper_t<float, at::native::sum_functor<float, float, float>::operator()(at::TensorIterator&):: {lambda(float, float)#1}>, unsigned int, float, 4, 4>) [clone .kd]	6 / 6	0.003759	0.003799

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
drafter / Drafter void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunctionOnSelf_add<long>, std::array<char*, 2ul> >(int, at::native::CUDAFunctionOnSelf_add<long>, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.001732	0.001866
drafter / Drafter void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda(float)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda(float)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.002479	0.002486
drafter / Drafter void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda(c10::BFloat16)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda(c10::BFloat16)#1}, std::array<char*, 2ul>) [clone .kd]	12 / 12	0.003785	0.003858
drafter / Drafter void at::native::vectorized_elementwise_kernel<8, at::native::FillFunction<c10::BFloat16>, std::array<char*, 1ul> >(int, at::native::FillFunction<c10::BFloat16>, std::array<char*, 1ul>) [clone .kd]	12 / 12	0.005351	0.005218
drafter / Drafter void at::native::vectorized_gather_kernel<16, long>(char*, char*, long*, int, long, long, long, long, bool) [clone .kd]	6 / 6	0.002099	0.002139
drafter / Drafter void at::native::warpMergeSortKVInPlace<2, -1, 128, 16, float, long, at::native::GTOp<float, true>, unsigned int, 32>(at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, unsigned int, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, at::native::GTOp<float, true>, float) [clone .kd]	6 / 6	0.004952	0.005019
drafter / Drafter void r4d_gemm_w4a16_nt_m64_kernel<1, 1, false>(unsigned short const*, unsigned int const*, unsigned int const*, __hip_bfloat16*, int, int, int, int, int) [clone .kd]	6 / 6	0.004506	0.004599
drafter / Drafter void r4d_gemm_w4a16_nt_m64_kernel<1, 1, true>(unsigned short const*, unsigned int const*, unsigned int const*, __hip_bfloat16*, int, int, int, int, int) [clone .kd]	60 / 60	0.080050	0.081410
drafter / Drafter void vllm::rms_norm_kernel<c10::BFloat16, 8, 2, true>(c10::BFloat16*, c10::BFloat16 const*, long, long, long, long, long, c10::BFloat16 const*, long, float, int, int) [clone .kd]	6 / 6	0.003059	0.003192
drafter / Drafter void vllm::rms_norm_kernel<c10::BFloat16, 8, 4, true>(c10::BFloat16*, c10::BFloat16 const*, long, long, long, long, long, c10::BFloat16 const*, long, float, int, int) [clone .kd]	6 / 6	0.002752	0.002779
drafter / Drafter void vllm::rotary_embedding_kernel<c10::BFloat16, c10::BFloat16, true>(long const*, c10::BFloat16*, c10::BFloat16*, c10::BFloat16 const*, int, long, long, long, int, int, int, long, bool) [clone .kd]	6 / 6	0.002499	0.002579
target_body / Attention KV write reshape_and_cache_kernel_flash.kd	96 / 96	0.046324	0.046878
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_1.kd	0 / 96	0.000000	0.024404
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_3.kd	0 / 96	0.000000	0.027984
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_4.kd	0 / 96	0.000000	0.028471
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_6.kd	96 / 0	0.022437	0.000000

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_8.kd	96 / 0	0.030144	0.000000
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_arange_bitwise_and_eq_index_lt_remainder_select_split_where_2.kd	0 / 96	0.000000	0.029938
target_body / Attention Q/K normalization, RoPE and layout triton_red_fused_7.kd	96 / 0	0.026091	0.000000
target_body / Attention Q/K normalization, RoPE and layout void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1} const&:: {lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1} const&:: {lambda(int, bool)#1}) [clone .kd]	0 / 96	0.000000	0.033671
target_body / Attention Q/K normalization, RoPE and layout void stock_m1_gemma_norm<false, 256, 32>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 96	0.000000	0.050797
target_body / Attention Q/K normalization, RoPE and layout void stock_m1_gemma_norm<false, 256, 64>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 96	0.000000	0.040311
target_body / Attention decode void qwen_stock_m1_shared_decode<4, 16, 256, 6, 16, 0, 3430971>(R4DArgs, int) [clone .kd]	0 / 96	0.000000	5.285290
target_body / Attention decode void r4d_attn_decode_kernel<3, 16, 256, 6, 16, 0, 3430971>(R4DArgs, int) [clone .kd]	96 / 0	4.224365	0.000000
target_body / Attention input activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	96 / 96	0.042844	0.042158
target_body / Attention input projection void radiancexfp4_fp8_gemm_decode<8, 128, 1, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	96 / 96	1.099262	1.098923
target_body / Attention output activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	96 / 96	0.045851	0.045791
target_body / Attention output gating triton_poi_fused_mul_mxfp4_linear_sigmoid_view_0.kd	96 / 96	0.028144	0.032364
target_body / Attention output projection void radiancexfp4_fp8_gemm_decode<8, 128, 4, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	96 / 96	0.556920	0.521433
target_body / Attention split-KV merge void qwen_stock_m1_shared_merge<256, 4, 1>(R4DArgs, int, int) [clone .kd]	0 / 96	0.000000	0.119631
target_body / Attention split-KV merge void r4d_attn_splitkv_combine_kernel<256, 4, 1>(R4DArgs, int, int) [clone .kd]	96 / 0	0.137671	0.000000

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
target_body / Embedding + first input normalization triton_poi_fused__to_copy_embedding_0.kd	0 / 6	0.000000	0.002599
target_body / Embedding + first input normalization triton_red_fused__to_copy_add_embedding_mxfp4_linear_rms_norm_0.kd	6 / 0	0.004112	0.000000
target_body / Embedding + first input normalization void stock_m1_gemma_norm<false, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 6	0.000000	0.005159
target_body / Final normalization/layout triton_red_fused__to_copy_add_fused_add_rms_norm_3.kd	6 / 0	0.002392	0.000000
target_body / Final normalization/layout void stock_m1_gemma_norm<true, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 6	0.000000	0.005612
target_body / GDN convolution _causal_conv1d_update_kernel.kd	0 / 288	0.000000	0.226365
target_body / GDN convolution void r4d_gdn_conv_update_kernel<1>(unsigned short const*, long, unsigned short const*, unsigned short const*, unsigned short*, long, long, long, int, int const*, long, int const*, unsigned short*, unsigned short*, unsigned short*, int const*, int, int, int) [clone .kd]	288 / 0	0.494015	0.000000
target_body / GDN input activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	288 / 288	0.127500	0.127346
target_body / GDN input projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 1, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	288 / 288	3.875011	3.872797
target_body / GDN layout/copies and buffer initialization triton_per_fused_1.kd	96 / 0	0.022318	0.000000
target_body / GDN layout/copies and buffer initialization triton_poi_fused_0.kd	96 / 0	0.031257	0.000000
target_body / GDN layout/copies and buffer initialization triton_poi_fused_add_1.kd	0 / 18	0.000000	0.004004
target_body / GDN layout/copies and buffer initialization triton_poi_fused_add_2.kd	0 / 12	0.000000	0.002738
target_body / GDN layout/copies and buffer initialization void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	288 / 288	0.156100	0.157119
target_body / GDN layout/copies and buffer initialization void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda(float)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda(float)#1}, std::array<char*, 2ul>) [clone .kd]	0 / 288	0.000000	0.092759

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
target_body / GDN layout/copies and buffer initialization void at::native::vectorized_elementwise_kernel<8, at::native::FillFunctor<c10::BFloat16>, std::array<char*, 1ul> >(int, at::native::FillFunctor<c10::BFloat16>, std::array<char*, 1ul>) [clone .kd]	288 / 288	0.074632	0.074886
target_body / GDN output activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	288 / 288	0.128079	0.131086
target_body / GDN output gated normalization layer_norm_fwd_kernel.kd	0 / 288	0.000000	0.097307
target_body / GDN output gated normalization triton_per_fused__to_copy_mean_pow_view_0.kd	192 / 0	0.050375	0.000000
target_body / GDN output gated normalization triton_poi_fused__to_copy_add_mean_mul_mxfp4_linear_pow_rsqrtsilu_view_1.kd	192 / 0	0.058742	0.000000
target_body / GDN output gated normalization triton_poi_fused__to_copy_add_mean_mul_mxfp4_linear_pow_rsqrtsilu_view_2.kd	96 / 0	0.028797	0.000000
target_body / GDN output projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 4, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	288 / 288	1.884129	1.883793
target_body / GDN recurrence and gates stock_gdn_scan_kernel.kd	0 / 288	0.000000	1.218805
target_body / GDN recurrence and gates void r4d_gdn_recurrent_update_kernel<1, 0, 2>(unsigned short const*, unsigned short const*, unsigned short const*, void const*, void const*, long, float const*, float const*, float*, long, long, unsigned short*, int const*, int const*, long, int const*, unsigned short const*, float const*, float, int, int, int, float, float) [clone .kd]	288 / 0	1.161877	0.000000
target_body / Layer input residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_3.kd	90 / 0	0.037245	0.000000
target_body / Layer input residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_4.kd	192 / 0	0.078508	0.000000
target_body / Layer input residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_5.kd	96 / 0	0.039184	0.000000
target_body / Layer input residual/normalization void stock_m1_gemma_norm<true, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 378	0.000000	0.345972
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_0.kd	0 / 288	0.000000	0.141272
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_1.kd	0 / 96	0.000000	0.048204
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_2.kd	96 / 0	0.044918	0.000000
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_3.kd	192 / 0	0.066262	0.000000
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_4.kd	96 / 0	0.045138	0.000000
target_body / MLP down input FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	384 / 384	0.296558	0.297344

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
target_body / MLP down projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 4, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	384 / 384	5.147348	5.142933
target_body / MLP gate/up input FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	384 / 384	0.164856	0.166543
target_body / MLP gate/up projection void radiance_mxfp4_fp8_gemm_folded<2, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, std::bfloat16_t*, int, int, int) [clone .kd]	384 / 384	25.271205	25.437279
target_body / Post-attention/GDN residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_1.kd	96 / 0	0.035164	0.000000
target_body / Post-attention/GDN residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_2.kd	192 / 0	0.064875	0.000000
target_body / Post-attention/GDN residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_3.kd	96 / 0	0.042258	0.000000
target_body / Post-attention/GDN residual/normalization void stock_m1_gemma_norm<true, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 384	0.000000	0.352599
target_vocabulary_head / Full BF16 target head (anonymous namespace)::stock_m1_head_pair(__hip_bfloat16 const*, __hip_bfloat16 const*, __hip_bfloat16*) [clone .kd]	0 / 6	0.000000	4.139846
target_vocabulary_head / Full BF16 target head Cijk_Alik_Bljk_BBS_BH_Bias_HA_S_SAV_UserArgs_MT16x32x256_MI16x16x1_SN_LDSB0_AFC1_AG0_AGGSUA0_AGNTAB0_AFEM1_AFEM1_ASEM1_CD1_1_CLR1_CLS0_CADS0_DTLA0_DTLB0_DTLMO_DTVAA0_DTVB1_DTVMSA0_DTVMSB0_DTVSM0_DPLB0_EPS1_ELFLR0_EMLLn1_FDSI0_GRPm1_GRVWA8_GRVWB8_GSUAMB_GLS0_HPLR0_ISA1201_ICIW0_IU1_K1_LDSTI0_LBSPPA512_LBSPPB0_LBSPPMXSA0_LBSPPMXSB0_LBSPPM0_LPA16_LPB0_LPMXSA0_LPMXSB0_LPM0_LRVW8_LWPMn1_MIAV1_MIWT1_1_MXLIBL_MXSFNS_M040_MGRIPM1_NTN1_NTA0 NTB0_NTC0_NTD0_NTE0_NTMXSA0_NTMXSB0_NTM0_NTWS0_NVn1_NVA0_NVB0_NVC0_NVD0_NVE0_NVMXSA0_NVMXSB0_NVM0_NVWS0_NEPBS0_NLCA1_NLCB16_ONLL0_PAP0_PGL0_PGR1_PLR1_PKA0_SGROB0_SIA3_SS1_SPO0_SRVW0_SSO0_SVW1_SK0_SKFTR0_SKFDP00_SKXCCM0_SNLL0_SIP1_SGRO0_TDMI0_TDMIM0_TDMS0_TIN0_THn1_THA0_THB0_THC0_THD0_THE0_THMXSA0_THMXSB0_THM0_THWS0_TLDS1_TLDSM1_ULSGRO0_USL1_USLMX0_UIOFGRO0_UPLRP0_USFGR0n1_USI0_VSn1_VWA1_VWB1_WSGRA0_WSGRB0_WS32_WG16_4_1.kd	6 / 0	4.024325	0.000000
unattributed / Other GPU bookkeeping __amd_rocclr_copyBuffer.kd	172 / 172	0.075278	0.076165
unattributed / Other GPU bookkeeping __amd_rocclr_fillBufferAligned.kd	6 / 6	0.002699	0.002519
unattributed / Other GPU bookkeeping _combine_sampled_and_draft_tokens_kernel.kd	7 / 7	0.003432	0.003485
unattributed / Other GPU bookkeeping _compute_local_logits_stats_kernel.kd	6 / 6	0.028659	0.028819
unattributed / Other GPU bookkeeping _compute_slot_mappings_kernel.kd	7 / 7	0.003386	0.003485
unattributed / Other GPU bookkeeping _expand_idx_mapping_kernel.kd	7 / 7	0.002185	0.002339
unattributed / Other GPU bookkeeping _gather_block_tables_kernel.kd	7 / 7	0.005632	0.005852
unattributed / Other GPU bookkeeping _get_num_sampled_and_rejected_kernel.kd	6 / 6	0.002632	0.002792
unattributed / Other GPU bookkeeping _insert_resampled_kernel.kd	6 / 6	0.003466	0.003499

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
unattributed / Other GPU bookkeeping _post_update_kernel.kd	6 / 6	0.005292	0.005206
unattributed / Other GPU bookkeeping _prepare_pos_seq_lens_kernel.kd	7 / 7	0.002499	0.002392
unattributed / Other GPU bookkeeping _prepare_rope_positions_kernel.kd	7 / 7	0.003119	0.003232
unattributed / Other GPU bookkeeping _rejection_kernel.kd	6 / 6	0.007559	0.007219
unattributed / Other GPU bookkeeping _resample_kernel.kd	6 / 6	0.014159	0.015366
unattributed / Other GPU bookkeeping _scatter_num_accepted_kernel.kd	6 / 6	0.001939	0.001986
unattributed / Other GPU bookkeeping postprocess_mamba_fused_kernel.kd	6 / 6	0.002552	0.002699
unattributed / Other GPU bookkeeping precopy_mamba_align_fused_kernel.kd	7 / 7	0.003079	0.003099
unattributed / Other GPU bookkeeping preprocess_mamba_align_fused_kernel.kd	7 / 7	0.003119	0.003012
unattributed / Other GPU bookkeeping void (anonymous namespace)::elementwise_kernel_with_index<int, at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&)::{lambda()#1}::operator()() const:: {lambda()#3}::operator()() const:: {lambda(long)#1}>(int, at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&)::{lambda()#1}::operator()() const:: {lambda()#3}::operator()() const:: {lambda(long)#1}, function_traits<at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&)::{lambda()#1}::operator()() const:: {lambda()#3}::operator()() const:: {lambda(long)#1}>::result_type*) [clone .kd]	42 / 42	0.010240	0.014853
unattributed / Other GPU bookkeeping void (anonymous namespace)::softmax_warp_forward<float, float, float, 6, false, false, 32>(float*, float const*, int, int, int, bool const*, int, bool) [clone .kd]	6 / 6	0.002226	0.002352
unattributed / Other GPU bookkeeping void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}) [clone .kd]	48 / 48	0.015452	0.015559
unattributed / Other GPU bookkeeping void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}) [clone .kd]	6 / 6	0.002579	0.003046

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
unattributed / Other GPU bookkeeping void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const::{lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const::{lambda(long)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const::{lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const::{lambda(long)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	48 / 48	0.020612	0.020886
unattributed / Other GPU bookkeeping void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl_nocast<at::native::CUDAFunction_add<int> >(at::TensorIteratorBase&, at::native::CUDAFunction_add<int> const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::CUDAFunction_add<int> >(at::TensorIteratorBase&, at::native::CUDAFunction_add<int> const&)::{lambda(int, bool)#1}) [clone .kd]	42 / 42	0.021400	0.018933
unattributed / Other GPU bookkeeping void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::(anonymous namespace)::CompareFunctor<float> >(at::TensorIteratorBase&, at::native::(anonymous namespace)::CompareFunctor<float> const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::(anonymous namespace)::CompareFunctor<float> >(at::TensorIteratorBase&, at::native::(anonymous namespace)::CompareFunctor<float> const&)::{lambda(int, bool)#1}) [clone .kd]	18 / 18	0.018944	0.019577
unattributed / Other GPU bookkeeping void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	109 / 109	0.053089	0.053589
unattributed / Other GPU bookkeeping void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<8 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<8 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<8 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<8 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	12 / 12	0.005945	0.005985

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
unattributed / Other GPU bookkeeping void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_put_kernel_impl<at::native::OpaqueTy pe<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_put_kernel_impl<at::native::OpaqueType<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_put_kernel_impl<at::native::OpaqueTy pe<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_put_kernel_impl<at::native::OpaqueType<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	6 / 6	0.003059	0.002979
unattributed / Other GPU bookkeeping void at::native::mbtopk::computeBlockDigitCounts<float, unsigned int, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int*, unsigned int, unsigned int, int, int, unsigned int, unsigned int, unsigned int*, short*) [clone .kd]	24 / 24	0.050263	0.052110
unattributed / Other GPU bookkeeping void at::native::mbtopk::computeBlockwiseWithinKCounts<unsigned int, float>(unsigned int*, short*, unsigned int*, unsigned int, int, bool, unsigned int*, float*, unsigned int*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	24 / 24	0.037523	0.038716
unattributed / Other GPU bookkeeping void at::native::mbtopk::fill<unsigned int, unsigned int>(unsigned int*, unsigned int, unsigned int) [clone .kd]	6 / 6	0.001612	0.001706
unattributed / Other GPU bookkeeping void at::native::mbtopk::gatherTopK<float, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int, bool, unsigned int, unsigned int, at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, unsigned int, unsigned int, float*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	6 / 6	0.026179	0.026026
unattributed / Other GPU bookkeeping void at::native::tensor_kernel_scan_innermost_dim<float, std::plus<float> >(float*, float const*, unsigned int, unsigned int, unsigned int, float, std::plus<float>) [clone .kd]	6 / 6	0.002646	0.002726
unattributed / Other GPU bookkeeping void at::native::unrolled_elementwise_kernel<at::native::CUDAFunction_add<int>, std::array<char*, 3ul>, 4, TrivialOffsetCalculator<2, unsigned int>, TrivialOffsetCalculator<1, unsigned int>, at::native::memory::LoadWithoutCast, at::native::memory::StoreWithoutCast>(int, at::native::CUDAFunction_add<int>, std::array<char*, 3ul>, TrivialOffsetCalculator<2, unsigned int>, TrivialOffsetCalculator<1, unsigned int>, at::native::memory::LoadWithoutCast, at::native::memory::StoreWithoutCast) [clone .kd]	42 / 42	0.012020	0.012020
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<16, at::native::BinaryFunction<bool, bool, bool, at::native::BitwiseOrFunction<bool> >, std::array<char*, 3ul> >(int, at::native::BinaryFunction<bool, bool, bool, at::native::BitwiseOrFunction<bool> >, std::array<char*, 3ul>) [clone .kd]	6 / 6	0.002759	0.002659
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<16, at::native::FillFunction<bool>, std::array<char*, 1ul> >(int, at::native::FillFunction<bool>, std::array<char*, 1ul>) [clone .kd]	7 / 7	0.002119	0.002185
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<16, at::native::bitwise_not_kernel_cuda(at::TensorIteratorBase&)::{lambda(bool)#1}, std::array<char*, 2ul> >(int, at::native::bitwise_not_kernel_cuda(at::TensorIteratorBase&)::{lambda(bool)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.002286	0.002332

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits)::{lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int)#1}, std::array<char*, 2ul> >(int, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits):: {lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int)#1}, std::array<char*, 2ul>) [clone .kd]	42 / 42	0.012046	0.014173
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits)::{lambda()}#1::operator()() const:: {lambda()}#4::operator()() const:: {lambda(long)#1}, std::array<char*, 2ul> >(int, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits):: {lambda()}#1::operator()() const:: {lambda()}#4::operator()() const:: {lambda(long)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.001772	0.001859
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&):: {lambda()}#1::operator()() const:: {lambda()}#7::operator()() const:: {lambda(float, bool)#1}, std::array<char*, 3ul> >(int, at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&):: {lambda()}#1::operator()() const:: {lambda()}#7::operator()() const:: {lambda(float, bool)#1}, std::array<char*, 3ul>) [clone .kd]	6 / 6	0.010073	0.010066
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::where_kernel_impl(at::TensorIterator&)::{lambda()}#1::operator()() const:: {lambda()}#11::operator()() const:: {lambda(bool, float, float)#1}, std::array<char*, 4ul> >(int, at::native::(anonymous namespace)::where_kernel_impl(at::TensorIterator&)::{lambda()}#1::operator()() const:: {lambda()}#11::operator()() const:: {lambda(bool, float, float)#1}, std::array<char*, 4ul>) [clone .kd]	12 / 12	0.004605	0.004878
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::BUnaryFunctor<int, int, int, at::native::binary_internal::div_floor_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int, int)#1}>, std::array<char*, 2ul> >(int, at::native::BUnaryFunctor<int, int, int, at::native::binary_internal::div_floor_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int, int)#1}>, std::array<char*, 2ul>) [clone .kd]	42 / 42	0.019407	0.015160
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::CUDataFunctorOnSelf_add<int>, std::array<char*, 2ul> >(int, at::native::CUDataFunctorOnSelf_add<int>, std::array<char*, 2ul>) [clone .kd]	48 / 48	0.013725	0.016085
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::CUDataFunctorOnSelf_add<long>, std::array<char*, 2ul> >(int, at::native::CUDataFunctorOnSelf_add<long>, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.001966	0.001899
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::CUDataFunctor_add<float>, std::array<char*, 3ul> >(int, at::native::CUDataFunctor_add<float>, std::array<char*, 3ul>) [clone .kd]	6 / 6	0.001939	0.001992
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::FillFunctor<float>, std::array<char*, 1ul> >(int, at::native::FillFunctor<float>, std::array<char*, 1ul>) [clone .kd]	12 / 12	0.003618	0.003111

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Final fixed ms per round
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::FillFunctor<int>, std::array<char*, 1ul> >(int, at::native::FillFunctor<int>, std::array<char*, 1ul>) [clone .kd]	7 / 7	0.001932	0.001886
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda(c10::BFloat16)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda(c10::BFloat16)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.010886	0.009939
unattributed / Other GPU bookkeeping void at::native::vectorized_gather_kernel<16, long>(char*, char*, long*, int, long, long, long, long, bool) [clone .kd]	6 / 6	0.002086	0.002252
unattributed / Other GPU bookkeeping void at::native::warpMergeSortKVInPlace<2, -1, 128, 16, float, long, at::native::GTOp<float, true>, unsigned int, 32>(at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, unsigned int, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, at::native::GTOp<float, true>, float) [clone .kd]	6 / 6	0.004832	0.004999

Prefill is outside this steady-decode timing table and has a separate causal/chunk-partition repair. CPU scheduling has no GPU-kernel duration here. The last row retains GPU bookkeeping outside the attributed model scopes; its individual kernels are listed without inventing missing semantic labels.

5. Brief unprofiled engine controls — 60K input, not 60K output

The approximately 80–87 tok/s figures are short controls on one 60,000-input- token Pi fixture. They are not results from generating 60,000 tokens.

Compiled configuration	Natural responses	Output tokens	Timed post-first seconds	Median round	Pooled post-first rate
Original compiled M8	3	2,582	30.142	59.684 ms	85.561 tok/s
Final compiled M8: Fix 1 + Fix 2	3	2,525	29.902	60.052 ms	84.342 tok/s

These fresh controls use the same builds as the stage profiles, in separate unprofiled passes. All three natural responses per arm are retained in the [profile and control audit](#).

These direct-engine controls exclude HTTP/Pi, tools, snapshot publication, cold prefill and warm-up. Rates pool tokens and time after the first output chunk; round latency is the median of the per-response steady-round medians. Throughput also depends on speculative acceptance, so it is not a pure kernel speed ratio. Clean timing passes do not record GPU profiles or copy activations.

Cost of preserving compiled BF16 rounding

Four fresh compiled runs used **before–after–after–before order**, with three natural responses per run on the same 60,000-token Pi prefix. Both settings use the final four performance repairs, full BF16 target head and piecewise GPU graphs. The only changed compiler setting is TORCHINDUCTOR_EMULATE_PRECISION_CASTS=0/1. The stage profiles in section 4 now include this rounding alignment. The following separate ABBA comparison isolates Fix 2 from Fix 1.

Compiled setting	Natural responses	Output tokens	Median round	Committed tokens/round	Pooled post-first rate
Before rounding alignment	6	4,474	60.906 ms	5.288	84.123 tok/s
BF16 intermediate casts preserved	6	5,050	60.900 ms	5.211	83.070 tok/s

Preserving casts changed the median round by **-0.006 ms (-0.01%)** and the pooled token rate by **-1.25%**. Mean committed tokens per round changed by -1.45%. The per-response median rounds ranged from 60.245 to 61.415 ms before and 60.567 to 61.194 ms after.

Round time is effectively unchanged in this sample. The slightly lower token rate is consistent with fewer committed tokens per round. Every run used the same sampling settings and three seeds, but changed rounding produced different natural continuations. The nearest-even native RoPE correction applies to the **eager reference**; this measures the compiled half of the alignment. Startup, compilation, warm-up and cold prefill are excluded from the decode timings. All twelve natural completions are included; per-round timings and source identities are retained in [rounding-speed-abba.json](#).

Separate, earlier 60K-generated-token verify-head experiment

This earlier compiled experiment used 115 natural responses per method across 11 intact Pi request boundaries, with 57,008–65,527 input tokens. It predates the final D7 repairs and **does not measure the final corrected M8's long-run speed**. The head methods also have different numerical guarantees.

Earlier head method	Natural responses	Generated tokens	Timed post-first seconds	Post-first tok/s
Full BF16 reference head	115	60,598	946.230	63.920
INT2 block-8 / rerank-80	115	60,075	892.548	67.178
INT2 global-128 / BF16 rerank	115	60,348	892.266	67.506
INT2 global-256 / BF16 rerank	115	60,675	906.873	66.779

The first output chunks are excluded from both the rate numerator and timed denominator. See [earlier-60k-output-speed.json](#).

6. Repairs and upstream ownership

1. **GDN prefill**: raw per-token gates and an ordered transition avoid the diagnosed dependence on future gates, cumulative-gate rounding and chunk partition. Core implementation belongs with libr4d; adapters belong in Radiance.
2. **GDN decode**: match serial convolution arithmetic, gate precision, recurrent reductions, output rounding and retained-prefix state. Rejecting a speculative suffix must not make its state persistent. Packed QKV views remove copies.
3. **Gated normalization**: retain the serial row tile without serializing all eight rows. The vendored FLA dispatcher belongs in vLLM.
4. **Attention**: retain each query's causal range and serial reduction/split policy; share KV reads to recover performance. Core ownership is libr4d.
5. **Residual/QK/final normalization**: preserve the chosen finite-precision boundaries; retain FP32 residual sums in registers. Existing vLLM rounding PRs must be checked for overlap before proposing another repair.
6. **Full BF16 head**: interleave two arithmetic-preserving M4 groups in one HIP launch. This avoids the first repair's duplicated launch/concatenation overhead. The upstream skinny-GEMM implementation belongs in vLLM.
7. **Fix 1 compiled integration**: install dispatch before tracing/capture, use opaque bindings where necessary, and validate startup-only fallbacks separately from real requests. Integration and conformance adapters belong in Radiance.
8. **Fix 2, execution-mode rounding**: enable TORCHINDUCTOR_EMULATE_PRECISION_CASTS=1 for the compiled target and preserve nearest-even BF16 products in native eager MRoPE. These changes address a separate eager/compiled discrepancy after Fix 1 had aligned M1 and M8.

These are submission scopes, not a claim that the pinned adapters apply unchanged to current upstream main. The new ports require their own tests. Upstream links and submission state are maintained in [submissions.md](#).

7. Execution-mode diagnosis and remaining limits

The eager/compiled mismatch was localized to intermediate BF16 casts and native RoPE multiplication. Aligning both produced exact eager/compiled **M8** output agreement on the controlled 320-position Pi replay. This is separate from the compiled M1/M8 10K result in section 3; the full 10K eager/compiled comparison has not been repeated with the new rounding settings.

The [complete boundary comparison](#) also matches captured inputs and outputs at all 465 matched boundaries, across 320 decode and nine sampled prefill positions. This sampled activation check supplements the whole-model and isolated-stage results; it does not cover all arbitrary inputs or every persistent state transition.

Intermediate BF16 casts

Before alignment, the first difference occurred in layer 0's MLP SiLU/gating, after identical GDN and gate/up projection outputs. The [isolated native replay](#) tested all 64 layers on common inputs: 20,480 decode layer-position evaluations and 576 sampled prefill evaluations. Each kernel reproduced its own captured output; none of the complete decode output rows agreed between modes.

```
eager:    BF16(BF16(SiLU(gate)) * up)
compiled: BF16(SiLU_FP32(gate) * up)
```

The BF16-intermediate arithmetic oracle matched eager in all 20,480 decode evaluations. An independent FP32-intermediate calculation still differed from the generated compiled kernel at 17 individual values, so that formula alone is not a bit-exact specification of the compiled operation.

Attention sigmoid gating had the same cast-elision distinction: eager rounded the sigmoid to BF16 before multiplying; compiled retained it in FP32. A [common-input native pilot](#) reproduced each implementation's own output and found 0/8 equal output rows between modes.

TORCHINDUCTOR_EMULATE_PRECISION_CASTS=1 preserves these intermediate casts. The experiment checks the loaded compiler boolean as well as the environment setting. Cast preservation alone did not restore equality, in the [cast-only control](#); it exposed the remaining RoPE difference after matching the first three layers.

Native RoPE multiplication

With compiler casts preserved, the first remaining discrepancy occurred in layer 3 after identical QKV projection and Q/K normalization. The [fine attention comparison](#) found:

Layer 3 boundary	Exact decode rows / 320	Exact sampled prefill rows / 9	Differing decode values
QKV projection	320	9	0
Query normalization	320	9	0
Key normalization	320	9	0
Query after RoPE	0	1	184,437
Key after RoPE	0	1	30,984
Value input	320	9	0
Attention output	0	1	1,335,594
Gate input	320	9	0
Gated attention output	0	1	1,282,380

The later output differences inherit changed rotary inputs and state; this table does not establish separate attention-kernel defects. On the same normalized Q/K and selected BF16 cosine/sine coefficients, the [explicit arithmetic oracle](#) reproduced each mode exactly at all 320 decode and nine sampled prefill positions:

```
native eager:    RNE_BF16(RTZ_BF16(a*c) - RTZ_BF16(b*s))
compiled emulation: RNE_BF16(RNE_BF16(a*c) - RNE_BF16(b*s))
```

The second rotary half uses addition with the same product-rounding rules; the 192 non-rotary coordinates are unchanged. RTZ means round toward zero; RNE means round to nearest, ties to even. This isolates rotation arithmetic, not the separate selection of cosine/sine coefficients.

The [native replay](#) then compared the installed kernel with a private copy that changed only product rounding:

Native RoPE variant	Q and K exact versus original eager	Q and K exact versus compiled cast preservation
Original native multiplication	320/320 decode; 9/9 prefill	0/320 decode; 1/9 prefill

Native RoPE variant	Q and K exact versus original eager	Q and K exact versus compiled cast preservation
Explicit nearest-even multiplication	0/320 decode; 1/9 prefill	320/320 decode; 9/9 prefill

Both Q and K independently meet those counts. Position zero is the identity rotation. Input/coefficient immutability and injected one-bit errors are checked. Source, LLVM IR, AMD assembly and GPU-binary hashes bind the result to the executed implementations; the full-model worker also verifies that the modified kernel runs during the request.

The installed **Triton 3.7.1 / PyTorch 2.12.0+rocm7.14** build lowers BF16 multiplication to `llvm.amdgcn.fdot2.bf16.bf16`, emitted as `v_dot2_bf16_bf16` on the R9700. Its observed truncation is the defect already fixed by [Triton PR #11227](#), merged August 14, 2026. That fix replaces the special lowering with an FP32 multiply and explicit nearest-even conversion. The new evidence here is the defect's occurrence and causal isolation inside the captured Qwen RoPE path; the upstream repair is credited to its existing authors.

Remaining coverage limits

- The rounding-aligned eager/compiled result covers 320 M8 decode positions and the final prefill prediction. Full 10K eager/compiled and M1/M8 qualification under these changed settings remains outstanding. The brief compiled speed comparison is reported in §5; eager execution speed has not been measured.
- Captured activation equality does not certify uncaptured KV/GDN/convolution state, unpaired operations or independent MRoPE coefficient selection.
- The original compiled 10K replay covers the all-seven-accepted D7 path. Small operator tests exercise rejection boundaries and injected faults, but full model coverage of every rejection history, long-context range, concurrency, snapshot restore and hardware platform is not established.
- Neither experiment proves arbitrary-input equivalence to an independent mathematical reference, eliminates model-generated loops or measures improved coding accuracy. Evidence remains tied to its source, binary and configuration identities; changed implementations require qualification.

8. Evidence and reproduction

The `evidence/` directory contains aggregate comparisons and kernel profiles, with file SHA-256 values in `evidence-sha256.json`. Recompute the stage table, kernel accounting and aggregate assertions without GPU use:

```
uv run python reports/d7-rdna4-2026-09-17/build_report.py
```

The original Pi transcripts, token IDs, ranked token IDs and raw traces remain private. The aggregate measurements can be audited from the published receipts; independent end-to-end reproduction needs a public replacement workload. Synthetic operator regressions accompany the corresponding source submissions.

Pinned arithmetic repair SHA-256: `8deef78cd58b355d85d142caac58a88ea68ad576a31fe7769cc3db49b18235e9`. Final performance bundle SHA-256: `d5e74aедda327d5543a0f4a47aeff80ec853c3eea5f9de49051b2d58ec109068`. 10K summary seal: `740f52b749130d91ecb870153a02e8b28c232489c87997423146c2af2d5496f8`.

References and attribution

- [Radiance](#), the qualified downstream stack.
- [libr4d](#), RDNA4 attention and recurrent kernels.
- [vLLM](#), including ROCm skinny GEMMs and vendored FLA.
- [Flash Linear Attention](#), serial GDN/normalization foundations.
- [vLLM batch invariance](#), related existing work.
- [PyTorch numerical accuracy](#), batched/reduction-order limitations.
- Existing normalization work: [vLLM #49639](#) and [#52243](#).

This contribution builds on those implementations. Batch invariance itself is not introduced by this report. Earlier DFlash RNG work in Radiance PR #8 backports [vLLM #54282](#) and is not presented as a new upstream discovery here.